B4 Polars and DuckDB

Tools for larger local datasets.

B4-L1 Polars expressions and lazy frames

Scan lazily, filter and select early, aggregate with expressions, and collect once.

```python
import polars as pl

shares = (
    df.group_by("wave", "brand")
    .agg(pl.col("weight").sum().alias("brand_weight"))
    .with_columns(total=pl.col("brand_weight").sum().over("wave"))
)
```

Polars eager vs lazy	DataFrame runs each step at once. LazyFrame (scan_*, .lazy()) builds a plan and runs on collect().
Group total on every row in Polars	pl.col("x").sum().over("group").
Polars sum() of an all-null group	0, not null. Report null_count() with totals.
Mean that skips nulls in Polars	pl.col("x").mean(). sum() / pl.len() counts null rows.

B4-L2 DuckDB, SQL over files and DataFrames

One connection passed in, parameters for values, and a None check on aggregates.

```python
import duckdb

con = duckdb.connect()   # in-memory database
rel = con.sql("""
    SELECT wave, brand, sum(weight) AS total
    FROM read_parquet('extracts/*.parquet')
    GROUP BY wave, brand
""")
df = rel.df()    # pandas; .pl() for Polars, .fetchall() for tuples
```

```python
con.execute("SELECT count(*) FROM responses WHERE wave = ?", [wave])
```

Query many Parquet files in DuckDB	FROM read_parquet('dir/*.parquet').
Pass a value into DuckDB SQL	con.execute("... WHERE x = ?", [value]). Never an f-string.
Table names from user input	Placeholders cannot bind identifiers. Check the name against an allowlist.
SQL sum over no rows	NULL, returned as None. Check with is None.

B4-L3 Choosing pandas, Polars, DuckDB or Spark

One primary engine. Reduce where the data is read. Convert small results at a clear boundary.

```python
lazy_result = pl.scan_parquet("extracts/
  *.parquet").filter(...).group_by(...).agg(...)
summary = lazy_result.collect().to_pandas()     # Polars to pandas for statsmodels
rel = con.sql("SELECT ... FROM read_parquet('extracts/*.parquet')")
top = rel.pl()                                  # DuckDB to Polars
```

Group-by over 10M rows: pandas, Polars, DuckDB	0.22 s, 0.11 s, 0.06 s on 4 cores. All fast enough. Choose by size against memory and where data lives.
Mixing engines	Reduce first in the engine that reads the files, convert the small result last.
pl.from_pandas and the index	Dropped by default. reset_index() first, or include_index=True.
When Spark	Data already in the lakehouse, larger than one machine, or a scheduled production job.